

Chapter 8

MPI: Message passing interface

All programs should start with "MPI_Init(&argc, &argv);" before any MPI calls

- Some examples online use MPI_Init(NULL, NULL) but it seems bad to not give arguments. Don't do this.

Before program closes, it should call "MPI_Finalize();"

- If this is not done, MPI runner will throw an error

Knowledge of world size and current rank is needed for most MPI programs

- `mpi_rank = MPI_Comm_rank(MPI_COMM_WORLD, &rank);`
- `world_size = MPI_Comm_size(MPI_COMM_WORLD, &nprocs);`

MPI Compilers are defined via wrappers to make it easier to compile

- `mpicc`: MPI + GCC
- `mpiicc`: MPI + ICC
- `mpicxx`: MPI + GCC (C++)
- `mpifort`: MPI + Fortran

Programs must be started with a startup command

- `mpirun`
- `mpiexec`
- `aprun`
- `Srun`

MPI involves communication commands that can be blocking or non-blocking

- `MPI_Send()`: blocking command that sends data, can block at bad times and get stuck
- `MPI_Recv()`: blocking command that receives data, can block at bad times and get stuck
- `MPI_Sendrecv()`: blocking command that sends and receives data, no problems with synchronization
- `MPI_Bcast()`: blocking command to distribute some data from a rank to all other ranks
- `MPI_Scatter()`: splits and distributes an array based on rank
- `MPI_Gather()`: collects and concatenates an array based on rank, maintains order by rank, only goes to rank 0
 - Can be used to reverse a "scatter"
- `MPI_Allgather()`: collects like `MPI_Gather`, but collects to all ranks
- `MPI_Reduce()`: reduces value from all ranks to rank 0
- `MPI_Allreduce()`: reduces value from all ranks to all ranks
- `MPI_Barrier()`: Forces MPI tasks to synchronize to same barrier
- More functions are defined in the standard

Avoiding hangs / deadlocks:

- Use combined collective calls rather than `MPI_recv` or `MPI_send` directly, to avoid blocking at a bad time

- If one task sends, another must recv it. This can be split as even/odd priority on sending or receiving first.

Non-blocking MPI:

- Non-blocking versions of some functions exist, allowing communication to happen while computations continue
 - `MPI_Request requests[2] = {MPI_REQUEST_NULL, MPI_REQUEST_NULL};`
 - `MPI_Irecv(xrecv, count, MPI_DOUBLE, partner_rank, tag, comm, &requests[0]);`
 - `MPI_Isend(xsend, count, MPI_DOUBLE, partner_rank, tag, comm, &requests[1]);`

Ghost cells: Special values on the edge of the computing region to ensure that all ranks are kept in sync. Similar to halo cells which are used to maintain boundary conditions, but these are updated as the loop occurs

There are special functions which allow for a cartesian grid to be used for distributing work (2D / 3D grid)

- This is helpful for setting up ghost cells without extra work
- This can also be used if the topology of the network is known

If combining MPI + OpenMP, changes should be made and thread safety must be considered.

- `MPI_Init_thread(&argc, &argv, int thread_model required, int *thread_model_provided);`
 - `MPI_THREAD_SINGLE`: Only one thread
 - `MPI_THREAD_FUNNELED`: Main thread does MPI
 - `MPI_THREAD_SERIALIZED`: One thread does MPI at a time
 - `MPI_THREAD_MULTIPLE`: All threads do MPI whenever